

QROUTER · ARCHITECTURE PROPOSAL · AUGUST 2026

The QRouter Encoding Layer

A verified account of how quantum programs are currently translated for six providers, why that translation is unsound, and a replacement architecture — the Quantum Execution Envelope.

Audit finding. The `ch` gate — a standard OpenQASM 2 `qelib1` gate — is decomposed into the wrong unitary. On the reproduction in §2.3 the platform returns the bitstring `11` with probability 1.0 where the correct answer is `01` with probability 1.0. No error is raised, and all 232 existing tests pass.

Version	qee-1.0.0-draft · proposal
Scope	everything between a submitted program and the bytes a provider executes
Baseline	commit 1503057 · 232 tests passing · typecheck clean
Method	every behavioural claim reproduced by executing the repository's own code

Contents

The QRouter Encoding Layer — Architecture, Evidence, and Migration Plan

- 0. Executive summary
- 1. How to read this

Part I — Audit

- 2. What the encoding layer is today

Part II — Research

- 3. What the providers actually require

Part III — The Quantum Execution Envelope

- 4. Design principles
- 5. The architecture

Part IV — Adversarial review

- 6. Round 1 — semantic correctness and cross-paradigm completeness
- 7. Round 2 — cost, complexity, performance, and migration feasibility
- 8. Round 3 — security, versioning, provider drift, and failure recovery
- 9. Rejected alternatives

Part V — Migration

- 10. Strangler rollout
- 11. What success looks like

Part VI — Appendices

- Appendix A — Full unitary audit of the lowering rules
- Appendix B — Reproducing the findings
- Appendix C — Defect index
- References

The QRouter Encoding Layer — Architecture, Evidence, and Migration Plan

Version: qee-1.0.0-draft · **Status:** proposal · **Scope:** everything between a submitted program and the bytes a provider actually executes

0. Executive summary

QRouter routes quantum workloads to IBM, IonQ, Rigetti, IQM, Amazon Braket, and its own simulator. Each of those targets speaks a different language, has a different instruction set, a different result schema, and a different bit-ordering convention. The component that reconciles them is the *encoding layer*. Today that component does not exist as a component. It exists as a QASM 2 string threaded through five files, rewritten by regular expressions at four points, and re-parsed by three different parsers that disagree with each other.

This document proposes replacing it with a **Quantum Execution Envelope (QEE)**: a versioned, content-addressed, cryptographically hashable structure with a discriminated workload union, a capability-satisfaction router, per-provider adapters that emit *exact executable artifacts*, and a typed result union with an explicit decoder.

The case for doing this is not aesthetic. During the audit for this document, a **provable silent-correctness bug** was found and reproduced: the `ch` gate — a *standard* OpenQASM 2 `qelib1` gate — is decomposed into the wrong unitary. On a circuit where the correct answer is the bitstring `01` with probability 1.0, QRouter returns `11` with probability 1.0. No error is raised. All 232 existing tests pass. Section 2.3 gives the reproduction.

That bug is not an isolated slip. It is what the current architecture makes likely, because:

- semantic correctness is asserted in a **comment**, not enforced by a check;
- the verification that does exist tests **one input state**, not the operator;
- the router decides **where** to run before anything proves the program **can** run there;
- results are coerced into `{counts, probabilities, shots}`, a shape that cannot represent what half the providers actually return.

Six defect classes were verified by execution, not inspection. Four more were verified by reading the call graph. They are catalogued in Section 2 with reproductions.

One framing correction, stated early. The brief asked for “the most advanced encoding algorithms for quantum.” There are two unrelated things called encoding, and conflating them would build the wrong system:

1. **Provider encoding** — translating one fixed program into the exact bytes a given backend executes. This *must be semantics-preserving*. Its virtue is fidelity, not cleverness.
2. **Application data encoding** — amplitude, angle, basis, block encoding, QSVT, qRAM. These *change the algorithm*. A router that silently applied one would be corrupting user intent.

The advanced algorithms belong to category 2 and are delivered as an **explicit, opt-in recipe library** (Section 5.12), never as an implicit router behaviour. Category 1 is where the engineering rigour goes. Section 4.6 argues this at length, because it is the single most important design decision in the document.

1. How to read this

Part I is an audit of the code as it stands, with reproductions. Part II is the standards and provider research. Part III is the proposed architecture. Part IV subjects that architecture to three adversarial review rounds and records what changed. Part V is the migration plan. Part VI holds appendices, including the full numerical audit table and reproduction instructions.

Every claim about the current code carries a file and line reference. Every claim about a provider carries a citation. Every claim about behaviour was produced by running the code.

Part I — Audit

2. What the encoding layer is today

2.1 The pipeline as built

```

source (OpenQASM 2 | OpenQASM 3 text)
|
|─ analyze.ts:15  toOpenQasm2()      regex downgrade 3 → 2; rejects on keyword scan
|─ dialects.ts:204 expandDialects()  text-splice rewrite to a "core" gate set
|─ analyze.ts:79  analyzeCircuit()   parse via quantum-circuit → CircuitAnalysis
|                                     (canonical form = normalizedQasm2: a STRING)
|
|─ route.ts:70    routeCircuit()     picks a backend from qubit count + price + queue
|
|─ transpiler.ts  transpileForBackend()
|                 |─ remote: services/simulator/app.py:262 compile_circuit()
|                 |   Qiskit transpile → qasm2 + qasm3 + QPY
|                 |─ local:  quantum-computer-js optimizer, or verbatim passthrough
|
|─ pipeline.ts:25 reprice against the compiled circuit
|
|─ execution.ts   submitToProvider()
|                 |─ Braket : qasm2ToQasm3()      regex, cx→cnot
|                 |─ IBM    : qasm2ToQasm3()      regex, stdgates
|                 |─ IonQ   : qasm2ToIonqCircuit() hand-written QASM parser → JSON
|                 |─ QCI    : normalizedQasm2 → worker
|
|─ results.ts:44  normalizeProviderResult() → {counts, probabilities, shots, metadata}

```

2.2 The three structural facts

Fact 1 — the canonical representation is a string. `CircuitAnalysis` ([types.ts:20](#)) carries `normalizedQasm2: string` plus eight scalar metrics. There is no AST. Every stage that needs structure re-derives it by regular expression or re-parses the text. Three different parsers are involved — `quantum-circuit` in analysis, `quantum-computer-js` in local optimization, Qiskit's `qasm2` in the worker — and they do not agree on what they accept.

Fact 2 — compilation output and execution input are disconnected. The Python worker compiles to a Qiskit circuit and serialises it three ways: QASM 2, QASM 3, and QPY ([app.py:298](#)). QPY is the only lossless one. `publicTranspilation` ([transpiler.ts:189](#)) then *deletes* `providerProgram` from the model, and `grep` confirms it is never read anywhere in `src/`, `cli/`, or `sdk/`. The worker even exposes `POST /v1/providers/ibm/jobs` ([app.py:435](#)) that accepts exactly that QPY and submits it via `SamplerV2`. **Nothing in the TypeScript codebase ever calls it.** IBM execution instead rebuilds a QASM 3 *string* by regex ([execution.ts:24](#)) and posts that. The lossless artifact is computed, discarded, and replaced with a lossy reconstruction.

Fact 3 — routing precedes proof of executability. `compatibility()` ([route.ts:16](#)) rejects a backend for: unavailability, offline status, `analysis.qubits > backend.qubits`, and the user's own cost/queue/fidelity/provider constraints. That is the entire capability model. It does not consult the gate set, the coupling map, classical-control support, measurement capability, or result types. A backend is declared compatible, quoted, and paid for; only afterwards does the encoder discover it cannot express the program.

2.3 Verified defects

Each entry below was reproduced by executing the repository's own code. Severity is by user-visible impact, not by effort to fix.

D1 — `ch` produces the wrong unitary. Silent. Standard gate. Critical.

`dialects.ts:156` decomposes controlled-Hadamard as:

```
ch: (_p, [a, b]) => [stmt("ry", [-PI / 4], [b]), stmt("cx", [], [a, b]), stmt("ry", [PI / 4], [b])],
```

Working through it in circuit order, the target-qubit operator conditioned on `control = 1` is $RY(\pi/4) \cdot X \cdot RY(-\pi/4)$, which evaluates to $(1/\sqrt{2}) \cdot \begin{bmatrix} -1 & 1 \\ 1 & 1 \end{bmatrix}$ — that is $(X - Z)/\sqrt{2}$. Controlled-Hadamard requires $(X + Z)/\sqrt{2}$. The two signs are swapped. The correct decomposition is `ry( $\pi/4$ )` before the `cx` and `ry( $-\pi/4$ )` after.

A full-unitary check against every one of the 22 decompositions in the file (Appendix A) confirms `ch` is the only wrong one, and confirms it to 3.2×10^{-13} against the *incorrect* matrix while sitting 1.41 away from the correct one.

Because H and $(X-Z)/\sqrt{2}$ differ only in sign structure, the error is a **relative phase** and is invisible in many measurements — which is precisely why it survived. It becomes a total inversion as soon as the target is in superposition:

```
OPENQASM 2.0;
include "qelib1.inc";
qreg q[2];
creg c[2];
x q[0];          // control = 1
h q[1];          // target in |+>
ch q[0],q[1];    // controlled-H maps |+> -> |0>
measure q -> c;
```

	outcome
Correct (controlled-H)	01 with probability 1.0
QRouter as shipped	11 with probability 1.0

The platform returns the opposite answer with full confidence and no warning. `ch` is defined in `qelib1.inc`; any ordinary IBM-flavoured OpenQASM 2 program using it is affected.

The header of `dialects.ts` states: “Every decomposition in this file is verified numerically against reference unitaries (up to global phase) in `tests/qrouter.test.ts`.” Three parts of that sentence are inaccurate. The tests are in `tests/dialects.test.ts`. They compare **state vectors from one fixed input state**, not unitaries. And they cover 6 of the 22 decompositions — `ch`, `cy`, `crz`, `cu3`, `csx`, `sx`, `sxdg`, `rxx`, `ryy`, `rzz`, `iswap`, and `ms` have no numerical test at all. The `ms` case is notable: the test named “expands IonQ-native `gpi/gpi2/ms`” asserts only `gpi` and `gpi2`.

This is the load-bearing argument of the whole document. 232 tests pass. Typecheck passes. A standard gate is silently wrong. Correctness that lives in a comment is not correctness.

D2 — Cross-provider bit-order divergence, unreconciled. Critical.

IonQ documents its result keys as **big-endian integers where the leftmost bit is qubit 0** — for a 3-qubit program, key `2` denotes `010` with `q0` on the left.¹

Qiskit and Aer use the opposite display convention: the **rightmost** character is qubit 0. The audit confirmed the same convention in `quantum-circuit`, the vendored simulator parser — `x q[0]` on `qreg q[2]` lands on state index 1.

`normalizeIonqProbabilities` ([execution.ts:212](#)) converts the integer to binary and left-pads it, preserving IonQ’s ordering. `normalizeProviderResult` ([results.ts:44](#)) then applies no reordering to anything.

The consequence: **the identical logical circuit returns bitstrings in reversed bit order depending on which backend the router happened to choose.** For an asymmetric circuit this is a wrong answer; for a symmetric one it is a coincidence. Nothing in the system records which convention a given result is in.

This is the defining failure mode of a router without an encoding layer: correctness becomes a function of routing, which is a function of price and queue depth.

D3 — Register boundaries are destroyed, and counts are silently merged. High.

Aer returns multi-register counts as space-separated fields (`"01 1"`). The worker stringifies them as-is ([app.py:357](#)). `normalizedStates` ([results.ts:25](#)) strips whitespace:

```
const clean = state.replace(/[>\s]/g, "");
```

Reproduced with `{"0 11": 50, "01 1": 50}`:

```
{ "counts": { "011": 50 }, "probabilities": { "011": 1 }, "shots": 50 }
```

Two physically distinct outcomes collapse to one key; one overwrites the other; **half the shots vanish** and the reported `shots` silently drops from 100 to 50, because `shots` is re-derived by summing the collapsed counts (`results.ts:52`).

The same class of bug exists on the IBM path. IBM returns one `BitArray` per classical register, keyed by register name.² `serialize_runtime_result` ([app.py:328](#)) iterates the registers and **adds their counts together into a single dictionary**. For a circuit with registers `meas` and `alpha`, the count reported for `"00"` is the sum of two unrelated measurements.

D4 — Quasi-probabilities are clipped; synthetic counts are unlabelled. High.

`normalizeProviderResult` accepts `quasiDistribution` and `quasi_dists` as a probability source. Quasi-probabilities from error mitigation are legitimately negative. `probabilitiesToCounts` ([results.ts:33](#)) does `Math.max(0, probability) * shots`.

Reproduced with `{"00": 1.2, "11": -0.2}` at 1000 shots:

```
{ "counts": { "00": 1200, "11": 0 }, "probabilities": { "00": 1.2, "11": -0.2 },
  "shots": 1000, "metadata": { "normalized": true } }
```

1200 counts from 1000 shots. The negative mass is discarded, the mitigation is silently undone, and `metadata.normalized: true` is the only annotation — there is no flag distinguishing counts that were *measured* from counts that were *synthesised by rounding a distribution*. Any downstream statistic computed from these counts is wrong in a way no consumer can detect.

D5 — The IonQ encoder silently drops the measurement map. High.

`qasm2ToIonqCircuit` ([execution.ts:99](#)) carries this docstring:

“Throws on anything it cannot faithfully express so a circuit is never silently altered before hardware execution.”

Line 135 skips `measure` and `barrier` via the same `continue` that skips the header:

```
if (/^(OPENQASM|include|qreg|creg|measure|barrier)\b/i.test(statement)) continue;
```

Reproduced on a 3-qubit circuit measuring only `q[0] → c[0]` and `q[2] → c[1]`, the encoder emits:

```
[ { "gate": "h", "target": 0 }, { "gate": "cnot", "control": 0, "target": 2 } ]
```

The submission then declares `qubits: 3` ([execution.ts:186](#)), IonQ measures all three, and the mapping from physical wire to the user’s classical bits — the thing that determines what the returned bitstrings *mean* — is gone. The docstring’s promise is violated by the function it documents.

D6 — Braket receives gate names Braket does not define. High.

`qasm2ToQasm3(source, "braket")` ([execution.ts:24](#)) renames `cx` → `cnot`, strips the `stdgates.inc` include, and passes everything else through. The “core” gate set that survives dialect expansion ([dialects.ts:15](#)) includes `sdg`, `tdg`, `u1`, `u2`, `u3`, `ccx`, and `id`.

Reproduced — the program actually submitted for a circuit using those gates:

```
OPENQASM 3.0;
qubit[3] q;
bit[3] c;
sdg q[0];
tdg q[1];
u3(0.1,0.2,0.3) q[2];
ccx q[0],q[1],q[2];
id q[0];
c = measure q;
```

Braket’s documented position is that a device’s supported gates come from its **device properties**, and that “no gate definitions are needed to use these gates” — the include is correctly unnecessary, but the *names* must be the ones the device publishes.³ Braket’s vocabulary uses `si` / `ti` rather than `sdg` / `tdg` and `ccnot` rather than `ccx`. This program is not valid for the target.

This path is reached whenever the remote compiler is unavailable and the target is a simulator, because `transpileForBackend` degrades to `localTranspile` ([transpiler.ts:166](#)) rather than failing — so `aws-sv1` with no compiler configured produces provider-rejected programs by construction.

The deeper error is the premise: there is **no universal core gate set**. `CORE_GATES` is a qelib1-flavoured set that no provider actually implements. Treating it as universal is what makes every downstream encoder a special case.

D7 — Advertised capability and encoder capability disagree. High.

The catalog gives `ionq-aria-1` these `basisGates` ([catalog.ts:44](#)):

```
["x","y","z","h","s","sdg","t","tdg","sx","sxdg","rx","ry","rz","cx","swap","measure"]
```

Those are handed to the Qiskit worker to build a `Target`, so the compiler is entitled to emit `sx` and `sxdg`. `qasm2ToIonqCircuit` has no case for either and throws `gate "sx" is not supported` ([execution.ts:159](#)) — at submission time, after routing, quoting, and fund reservation.

IonQ’s QIS gate set does contain these operations; they are named `v` and `vi`.⁴ The mapping simply was never written. The structural problem is that the capability declaration (`catalog.ts`), the compilation target (worker), and the encoder (`execution.ts`) are three independent hand-maintained lists with no mechanism forcing them to agree.

D8 — IBM ignores the routing decision. High.

```
async function submitIbm(analysis: CircuitAnalysis, shots: number): Promise<Submission> {
  ...
  backend: process.env.IBM_QUANTUM_BACKEND ?? "ibm_brisbane",
```

`submitIbm` ([execution.ts:69](#)) does not receive `backendId`. Whatever IBM backend the router selected, quoted, and priced, the job is submitted to the one named in an environment variable. The same is true of the worker's IBM path ([app.py:442](#)). The routing decision is not honoured, and the price the user was quoted may correspond to a different machine than the one that ran the job.

Separately, IBM requires that submitted circuits already conform to the backend's ISA — “circuits must adhere to the backend's ISA,” consisting only of Target-supported instructions and respecting connectivity.⁵ The QASM2 → text → QASM3 round trip in the current path provides no such guarantee, and the layout that would make it checkable is dropped (D9).

D9 — Layout and routing permutation are computed, then dropped. High.

`serialize_layout` ([app.py:172](#)) returns `logicalToPhysical` and `routingPermutation`. It reaches TypeScript as `TranspilationResult.layout` and is stored for display. It is never applied when decoding results.

When the transpiler routes a circuit, virtual qubit k generally does not end up on physical qubit k . Without applying the inverse permutation at decode time, returned bitstrings are indexed by *physical* position while the user reads them as *virtual* position. This is a silent relabelling of every measurement outcome on any hardware target with non-trivial routing — which is all of them.

D10 — Test-environment API keys can reach a QPU on `/api/v2`. Critical (authorization).

`scopes.ts` provides `backendsForPrincipal` and `assertTargetAllowed` ([scopes.ts:60](#)) precisely to enforce the documented rule that `qci_test_...` keys are simulator-only. A call-site census shows they are applied in `/api/v1/jobs`, `/api/v1/repository-jobs`, and `/api/v1/session` — and **nowhere under `/api/v2`**. `createExecutionGroup` → `prepareGroupExecutions` ([v2-service.ts:258](#)) passes `context.backends` through unfiltered.

A test key on the v2 API can therefore select, be quoted for, and execute on physical hardware, billing real credits. This is a live authorization gap independent of the encoding work, and it should be fixed before anything else in this plan.

D11 — Every backend advertises identical capabilities, including the disabled photonic ones. Medium.

`/api/v2/backends` ([route.ts:17](#)) stamps a constant onto every entry:

```
capabilities: { input_formats: ["openqasm2","openqasm3"], execution: "async",
               result: ["counts","probabilities","shots"] }
```

xanadu-borealis and quandela-mosaic are photonic devices whose own catalog entries carry `capabilityNote: "photonic backend requires a native-input bridge; gate-model circuits cannot be translated automatically"`. The API tells clients they accept OpenQASM. The capability document and the capability reality are unrelated objects.

D12 — OpenQASM 3 is handled by regex downgrade and a keyword blocklist. Medium.

`toOpenQasm2` ([analyze.ts:15](#)) rejects any program matching `\b(def|defcal|cal|while|for|switch|input|output|duration|stretch)\b` and otherwise rewrites declarations with six substitutions. Two consequences:

- The blocklist matches **identifiers and comment text**, so a register named `input_state` or a comment mentioning “for” is rejected as an unsupported construct.
- Everything OpenQASM 3 exists *for* — classical control flow, timing, pulse-level `defcal`, parameterised `input` — is unrepresentable, so the platform cannot express dynamic circuits at all, on any backend, regardless of whether that backend supports them.

D13 — The compiler worker response is spread into the model without validation. Medium.

```
return { ...(data as unknown as Omit<TranspilationResult, "backendId"|"compiler">), ... }
```

`transpiler.ts:146` casts an arbitrary JSON body straight into the typed model. Note the type disagreement this hides: `TranspilationResult.providerProgram` is declared `string | undefined` ([types.ts:108](#)) while the worker sends `{format, data}` ([app.py:303](#)). The mismatch is invisible because nothing reads the field.

D14 — Dialect expansion does not recurse into user-defined gate bodies. Medium.

`expandDialects` copies `gate name(...) { ... }` blocks verbatim and leaves calls to user-defined gates untouched ([dialects.ts:218](#)). A body containing a provider-native gate — `gate myecr a,b { ecr a,b; }` — passes through unexpanded and reaches downstream consumers that cannot interpret it.

D15 — `equivalent: boolean | null` is an ambiguous verification result. Medium.

The worker returns `None` for “*verification was not attempted*” (over 10 qubits), “*dimensions differ due to ancillas*”, and “*verification threw*” — three materially different situations ([app.py:205](#)). All arrive as `null`. A consumer cannot distinguish “unproven” from “unprovable” from “crashed”.

D16 — SQLite connections are never closed in the worker. Low (operational).

Every helper in `app.py` opens a connection via `with database()` as `connection`. `sqlite3`'s context manager commits the transaction; it does **not** close the connection. Under sustained load this leaks file descriptors.

D17 — Filesystem `include` reaches the worker's parser. Low — currently mitigated, not defended.

`expandDialects` passes `include` statements through untouched ([dialects.ts:239](#)), so a user-supplied `include` `"../../etc/passwd"` survives to the payload sent to the Qiskit worker, whose `qasm2.loads` resolves includes from the filesystem.

Reproduction shows the request is in fact **rejected** — `quantum-circuit` fails to parse the unknown `include` and `analyzeCircuit` raises `CircuitValidationError` before the worker is reached. So this is not currently exploitable. It is listed because the protection is **incidental**: it comes from a vendored parser's error behaviour, not from a policy. Any change to the analysis parser — including the AST migration proposed here — removes the accident. The mitigation must become explicit before the parser is replaced.

2.4 Root causes

The seventeen defects reduce to five structural causes. Each one maps to a specific piece of the proposed architecture, and the mapping is the argument for that piece.

#	Root cause	Defects	Addressed by
RC1	The canonical form is a string, so structure is re-derived by regex at every stage	D1, D3, D6, D12, D14, D17	§5.4 Semantic IR
RC2	There is no capability model; compatibility is a qubit-count comparison	D6, D7, D8, D11	§5.6–5.7 CapabilityProfile & satisfaction
RC3	The artifact that was compiled is not the artifact that is executed	D8, D9, D13	§5.9 Execution Bundle
RC4	Results are coerced into one shape that cannot hold what providers return	D2, D3, D4, D9	§5.10 Typed result union
RC5	Semantic correctness is asserted, not proven	D1, D15	§5.11 Verification portfolio

Note that D1 — the wrong gate — is caused by RC1 and *survived* because of RC5. Both halves need fixing; either alone would have left it shipping.

Part II — Research

3. What the providers actually require

The purpose of this section is to establish, from primary sources, what an encoding layer has to be able to say. The recurring finding is that provider surfaces differ not in syntax but in **expressive power** — and that no single interchange format covers them.

3.1 OpenQASM 3 as the semantic surface for gate programs

OpenQASM 3 is the only open specification that covers the gate-model workloads QRouter routes today *and* the dynamic and pulse-level features it cannot currently express. Beyond OpenQASM 2 it adds classical types and expressions, control flow (`if / else` , `for` , `while` , `switch`), timing (`duration` , `stretch` , `delay` , `barrier`), gate modifiers (`ctrl @` , `inv @` , `pow @`), subroutines (`def`), arrays, parameterised input / output , and pulse-level `cal / defcal` blocks.⁶

Two properties make it the right *source* language and a poor *universal target*:

- It is a **superset of what any single device runs**. Braket accepts `defcal` , `cal` , frames and waveforms in OpenPulse — but classical control, custom gates, and subroutines are documented as **LocalSimulator-only**, not available on QPUs or on-demand simulators.⁷ Support is per-device, not per-language.
- It has **no canonical serialisation**. Two textually different programs can be semantically identical, so the text cannot be hashed to identify a program.

Conclusion: parse OpenQASM 3 into a semantic AST and keep *that* as the gate-program IR. Do not keep the text, and do not try to round-trip through OpenQASM 2 — which is exactly what the current worker does (`qasm2.loads / qasm2.dumps` , `app.py:263, 301`), losing everything OpenQASM 3 adds.

3.2 QIR, and why it is not the canonical IR

QIR is an LLVM-based representation with profiles that define what a backend must support. The **Base Profile** requires that a program applies no instruction to a qubit after measuring it, that measurement happens only at the end, that qubit count is known at compile time, and that there is no dynamic qubit or result management.⁸

That is a precise description of a *static* circuit. It excludes mid-circuit measurement with feedback — the defining feature of dynamic circuits — which the Adaptive Profile exists to restore. And QIR has nothing to say about annealing, analog Hamiltonian simulation, or photonic workloads, which are not gate programs at all.

Conclusion: QIR is a valuable **lowering target** for hybrid and gate workloads headed to QIR-consuming backends. It is the wrong universal canonical IR, because adopting it would encode the Base Profile's restrictions into the platform's core data model.

3.3 Provider requirements, side by side

Gate-model targets.

	IBM (Runtime)	IonQ (v0.4 direct)	Braket (QPU / on-demand)	IQM (via Braket)
Program form	ISA circuit for the backend Target ⁹	QIS or native gate JSON; also <code>ionq.qasm3.v1</code> ¹⁰	OpenQASM 3 subset, device-specific gate names ¹¹	OpenQASM 3 via Braket; native <code>prx</code> / <code>cz</code> in a verbatim box
Native gates	<code>id</code> , <code>rz</code> , <code>sx</code> , <code>x</code> , <code>ecr</code>	<code>gpi</code> , <code>gpi2</code> , <code>ms</code> , <code>zz</code>	per device properties	<code>prx</code> , <code>cz</code>
Dynamic control	yes (Runtime dynamic circuits)	limited	LocalSimulator only	no
Pulse level	yes	no	OpenPulse <code>cal</code> / <code>defcal</code>	no
Result shape	<code>PrimitiveResult</code> → <code>PubResult</code> → <code>DataBin</code> , one <code>BitArray</code> per classical register ¹²	probability histogram; big-endian integer keys, leftmost bit = qubit 0 ¹³	counts, plus result-type pragmas: <code>probability</code> , <code>expectation</code> , <code>variance</code> , <code>sample</code> , <code>amplitude</code> , <code>state_vector</code> , <code>density_matrix</code> , <code>adjoint_gradient</code> ¹⁴	as Braket
Batch limits	per-PUB shots	≤ 5,000 circuits, ≤ 150,000 gates per job ¹⁵	per device	per device

Non-gate-model targets. These are not circuits at all, which is the point.

	D-Wave (annealing)	Photonic (Xanadu / Quandela)
Program form	QUBO / Ising / BQM / CQM	Fock / Gaussian / dual-rail; not gate-model
Native operations	n/a — an objective function, not a gate sequence	squeezing, displacement, beamsplitter
Dynamic control	n/a	n/a
Result shape	<code>SampleSet</code> : <code>sample</code> , energy , <code>num_occurrences</code> , <code>chain_break_fraction</code> ¹⁶	photon-number / click patterns
Batch limits	per solver	per device

Three observations follow directly from this table.

Observation 1 — `{counts, probabilities, shots}` is not a superset of any of these. It cannot hold D-Wave energies, Braket expectation values or gradients, IBM's per-register `BitArray`s, or photon patterns. Any layer that normalises to it is discarding information by design. This is D3 and D4 restated as an architectural fact rather than a bug.

Observation 2 — capability is per-device and dynamic, not per-provider and static. Braket says so explicitly: supported gates come from device properties. A hardcoded array in `catalog.ts` is a cached copy of a remote fact, with no refresh and no staleness signal.

Observation 3 — the workloads are not variants of one thing. A QUBO is not a circuit with different gates; it has no gates. Modelling everything as “a circuit, possibly with extensions” is how the photonic backends ended up permanently `available: false` with an apologetic note. This motivates the discriminated workload union (§5.3) over a single extensible circuit type.

3.4 Canonicalisation: choosing a hashing substrate

To content-address artifacts and sign capability profiles, a byte-exact serialisation is needed.

- **RFC 8785 (JCS)** canonicalises JSON: recursive lexicographic sort of property names by UTF-16 code unit, ECMAScript IEEE-754 number serialisation, defined string escaping, UTF-8 output. Input must be I-JSON — no duplicate keys, no `NaN / Infinity`, no numbers outside double range.¹⁷
- **RFC 8949 deterministic CBOR** gives a compact binary equivalent for large internal payloads.
- **Protobuf is explicitly not canonical** — the maintainers state serialisation is not deterministic across implementations or versions, so it must not be used as a hashing substrate.¹⁸

JCS’s number constraint is a genuine design input rather than a footnote: gate rotation angles are doubles, so parameters must be normalised to IEEE-754 doubles *before* hashing, and any symbolic parameter must be carried as a string expression rather than a float. Usefully, the existing `evaluateParam` already rejects non-finite results ([dialects.ts:90](#)), which is the same constraint arrived at independently.

Decision: JCS for the envelope and anything signed or externally visible; deterministic CBOR permitted for large internal blobs, with the JCS hash remaining the identity.

3.5 Equivalence checking

MQT QCEC provides four complementary strategies — construction, alternating, and simulation checkers over decision diagrams, plus a ZX-calculus checker — and supports compilation-flow verification, parameterised circuits, and partial equivalence.¹⁹ This matters because no single method scales: decision diagrams are excellent on structured circuits and can blow up on others; ZX is strong on Clifford+T; simulation-based checking is a fast falsifier that cannot prove equivalence.

The design consequence is that verification must be a **portfolio with an explicit status**, not a boolean. Exhaustive unitary comparison is affordable only for small circuits — the existing worker caps it at 10 qubits ([app.py:188](#)), which is a reasonable number for a dense $2^n \times 2^n$ operator and useless above it. Above that threshold the honest statuses are “checked by an independent method”, “provider-validated”, or “unproven” — never `null`.

3.6 The distinction that determines what to build

The brief asked for the most advanced encoding algorithms. Two different things share the name.

Provider encoding takes a fixed program and produces the exact bytes a backend executes. Its correctness criterion is *semantic preservation*. Nothing about it should be clever: the ideal provider encoder is boring, total, and provably faithful. D1, D5, and D6 are failures of this kind, and they are the ones costing users correct

answers today.

Application data encoding decides how classical information becomes a quantum state, and *changes the computation*:

Recipe	Cost	Honest status
Basis encoding	1 qubit per bit, trivial preparation	production-ready
Angle / rotation encoding	1 qubit per feature, depth $O(1)$	production-ready
Data re-uploading	repeated encoding layers, variational	production-ready, changes the model
Amplitude encoding	$\log_2 N$ qubits, but generic state preparation is exponential in gate count	opt-in with a computed and accepted cost
qRAM	assumes hardware that does not exist at scale	experimental / research only
Block encoding, QSVT	large ancilla and depth overhead	fault-tolerant regime; resource estimation
QUBO / Ising	native to annealers	its own workload family
Fock / Gaussian / dual-rail	native to photonics	its own workload family

The trap is that “advanced encoding layer” sounds like it means the second column. If the router chose amplitude encoding on the user’s behalf, it would silently substitute a different computation — with a state-preparation circuit whose depth can exceed the original algorithm — and then report results as if they answered the original question. That is a more severe version of the D1 failure mode: not a wrong gate, but a wrong algorithm.

Therefore: the encoding *layer* is category 1 and is held to a proof standard. Category 2 ships as an explicit, versioned **recipe library** (§5.12) that a user opts into per job, with its cost computed and surfaced before execution. Both are in the plan. Only one of them is allowed to happen implicitly.

Part III — The Quantum Execution Envelope

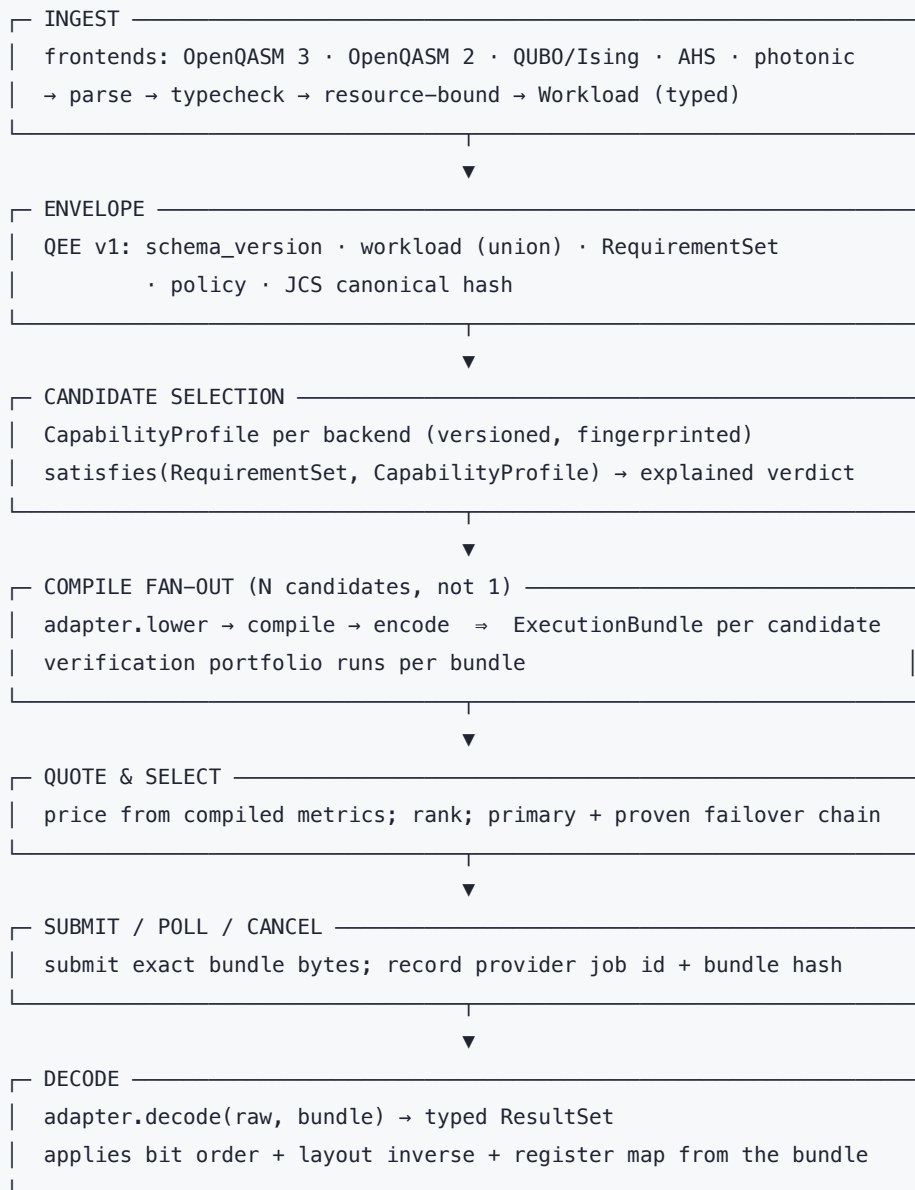
4. Design principles

Seven rules. Each is traceable to a defect in Part I; a design decision that does not serve one of these is out of scope.

1. **Nothing is a string.** Programs are structured values. Text exists only at the boundaries: once on ingest, once on emission. *(RC1)*
2. **Compile before you promise.** A backend is a candidate only after a target-bound artifact exists for it. Quoting an uncompiled backend is quoting a guess. *(RC2, RC3)*
3. **Execute the artifact you verified.** The bytes submitted are the bytes hashed, recorded, and checked — never a reconstruction. *(RC3)*
4. **Capabilities are declared by the code that implements them.** An adapter’s capability profile is derived from the adapter, not maintained beside it. *(RC2)*
5. **Results keep their shape.** Decoding is a typed, per-family operation that preserves register structure, bit order, and provenance. Synthesised values are labelled as synthesised. *(RC4)*
6. **Verification has a status, never a boolean.** “Unproven” is a first-class, reportable outcome. *(RC5)*
7. **Fail closed, loudly, early.** An encoder that cannot faithfully express a program refuses. It never approximates, and it never silently drops an instruction. *(D5)*

5. The architecture

5.1 Layer map



5.2 The envelope

```
interface ExecutionEnvelope {
  schema_version: "qee/1";
  id: string; // JCS-SHA-256 of everything below
  created_at: string;
  workload: Workload; // discriminated union, §5.3
  requirements: RequirementSet; // derived, §5.5
  policy: {
    routing_mode: "balanced" | "cost" | "speed" | "quality";
    constraints: RoutingConstraints;
    failover: { enabled: boolean; max_attempts: number };
    verification: { minimum_status: VerificationStatus };
  };
  provenance: {
    source_blob: BlobRef; // exact submitted bytes, content-addressed
    frontend: { name: string; version: string };
  };
}
```

`id` is the JCS canonical hash. Two submissions that mean the same thing produce the same `id` even if the text differed — which is what makes caching, idempotency, and deduplication correct rather than best-effort.

5.3 The workload union

The single most important decision in the design: **workloads are not variants of a circuit.**

```
type Workload =
| { kind: "gate";      program: GateProgram;  shots: number }
| { kind: "dynamic";   program: GateProgram;  shots: number } // classical feedback
| { kind: "timed";     program: TimedProgram;  shots: number } // duration/stretch/defcal
| { kind: "analog";    program: AhsProgram;    shots: number } // Hamiltonian evolution
| { kind: "annealing"; problem: QuboProblem | IsingProblem | CqmProblem; reads: number }
| { kind: "photonic";  program: PhotonicProgram; shots: number } // Fock/Gaussian/dual-rail
| { kind: "primitive"; pubs: Pub[] } // batched/parametric
| { kind: "estimation"; program: GateProgram; estimator: FtEstimatorConfig };
```

`gate` and `dynamic` share `GateProgram` but are separate variants because they have different *requirements*: `dynamic` demands mid-circuit measurement with feedback, which most targets do not have and which QIR’s Base Profile forbids outright.²⁰ Making that a `kind` rather than a flag means the router cannot forget to check it.

This is what unblocks the photonic backends. They stop being gate-model devices that “require a bridge” and become the correct targets for `photonic` workloads — a capability match rather than a permanent apology.

5.4 GateProgram : a semantic AST

```
interface GateProgram {
  qubits: QubitRegister[];           // named, sized, ordered – never flattened away
  clbits: ClassicalRegister[];
  params: ParamDecl[];               // OpenQASM 3 `input`
  body: Stmt[];
  gate_defs: Record<string, GateDef>; // resolved, with fully expanded bodies
}

type Stmt =
  | { op: "gate";   name: string; params: ParamExpr[]; qubits: QubitRef[];
    modifiers: Modifier[] }           // ctrl @ / inv @ / pow @
  | { op: "measure"; qubit: QubitRef; clbit: ClbitRef }
  | { op: "reset";   qubit: QubitRef }
  | { op: "barrier"; qubits: QubitRef[] }
  | { op: "delay";   duration: Duration; qubits: QubitRef[] }
  | { op: "if";      cond: BoolExpr; then: Stmt[]; else?: Stmt[] }
  | { op: "for";     var: string; range: RangeExpr; body: Stmt[] }
  | { op: "while";   cond: BoolExpr; body: Stmt[] }
  | { op: "switch";  subject: IntExpr; cases: SwitchCase[] }
  | { op: "call";    subroutine: string; args: Expr[] };
```

Four properties earn their keep:

- **Registers survive.** `qubits` / `clbits` stay named and sized end to end, so the decoder can reconstruct "011" rather than receiving "011" and guessing. Directly fixes D3.
- **The measurement map is data.** Every `measure` names its source qubit and destination clbit, so the map is a first-class object that can be carried into the bundle and applied at decode. Directly fixes D5 and D9.
- **Parameters are expressions, not floats.** A `ParamExpr` is either a normalised IEEE-754 double or a symbolic expression. This keeps JCS hashing well-defined (§3.4) while allowing parametric workloads.
- **Gate definitions are resolved.** `gate_defs` holds fully expanded bodies, so no consumer ever meets an unexpanded provider-native gate inside a user-defined block. Directly fixes D14.

On rewriting to a “core” gate set. The current design’s fatal premise is a universal core set (D6). In the new design there is no such thing. The IR carries whatever gates the source used, annotated with their definitions; **lowering to a gate set happens per target, inside the adapter, against that target’s declared capability profile.** There is exactly one intermediate vocabulary, and it belongs to the target, not to the platform.

Each decomposition used in lowering carries a machine-checkable reference unitary, and the build verifies every one of them (§5.11). D1 becomes impossible to reintroduce: a decomposition without a passing unitary proof does not compile.

5.5 RequirementSet

Derived from the workload; never hand-written.

```

interface RequirementSet {
  schema_version: "req/1";
  workload_kind: Workload["kind"];
  qubits: number;
  clbits: number;
  instructions: InstructionRequirement[]; // name + arity + parameter arity
  connectivity: { pairs: [number, number][]; needs_routing: boolean };
  classical: {
    mid_circuit_measurement: boolean;
    feedback: boolean; // measurement → gate dependency
    control_flow: ("if" | "for" | "while" | "switch")[];
  };
  timing: { explicit_delays: boolean; stretch: boolean; pulse_level: boolean };
  results: ResultTypeRequirement[]; // counts | expectation | gradient | energy | ...
  limits: { shots: number; depth: number; ops: number; batch_size: number };
}

```

5.6 CapabilityProfile

The counterpart, **produced by the adapter that implements it** (principle 4), refreshed from the provider, and fingerprinted.

```

interface CapabilityProfile {
  schema_version: "cap/1";
  backend_id: string;
  adapter: { name: string; version: string };
  source: "provider_api" | "static_fallback";
  fetched_at: string;
  staleness: { ttl_seconds: number; is_stale: boolean };

  workload_kinds: Workload["kind"][];
  instructions: InstructionCapability[]; // name, arity, param arity, qubit loci
  connectivity: { kind: "all-to-all" | "coupling_map"; coupling_map?: [number, number][] };
  classical: { /* mirrors RequirementSet.classical */ };
  timing: { /* mirrors RequirementSet.timing */ };
  result_types: ResultTypeCapability[];
  limits: { max_qubits: number; max_shots: number; max_depth?: number;
    max_ops?: number; max_batch?: number };

  calibration: { fingerprint: string; measured_at: string } | null;
  provider_schema_version: string;
  fingerprint: string; // JCS hash of everything above
}

```

The **fingerprint** is what makes drift detectable: an artifact records the profile fingerprint it was compiled against, so a capability change between compile and submit is a detectable condition rather than a mysterious provider rejection.

`source: "static_fallback"` is deliberately explicit. Today's `catalog.ts` arrays are static fallbacks pretending to be facts (D11, Observation 2). Naming them means the API can tell a client that a capability claim is a cached guess.

5.7 Compatibility as satisfaction

```
function satisfies(req: RequirementSet, cap: CapabilityProfile): Verdict;

type Verdict =
  | { ok: true; notes: string[] }
  | { ok: false; failures: Failure[] };    // each names the exact unmet requirement
```

Every check that D6, D7, D8 and D11 needed and did not have: workload kind, instruction coverage (by name *and* arity), connectivity satisfiability, classical-control support, timing support, result-type support, and every numeric limit. A rejection names what was missing, so `buildAlternatives` can explain the gap instead of reporting “requires 5 qubits; backend has 4”.

5.8 Adapter lifecycle

Every provider implements one interface. The lifecycle is the contract.

```
interface ProviderAdapter {
  discover(): Promise<BackendDescriptor[]>;
  profile(backend: BackendDescriptor): Promise<CapabilityProfile>;
  validate(env: ExecutionEnvelope, cap: CapabilityProfile): Verdict;
  lower(env: ExecutionEnvelope, cap: CapabilityProfile): Promise<LoweredProgram>;
  compile(lowered: LoweredProgram, cap: CapabilityProfile): Promise<CompiledProgram>;
  encode(compiled: CompiledProgram, cap: CapabilityProfile): Promise<ExecutionBundle>;
  submit(bundle: ExecutionBundle): Promise<ProviderHandle>;
  poll(handle: ProviderHandle): Promise<ProviderState>;
  cancel(handle: ProviderHandle): Promise<void>;
  decode(raw: unknown, bundle: ExecutionBundle): Promise<ResultSet>;
}
```

`encode` and `decode` are inverses over the same `ExecutionBundle`, which is what makes bit order and register mapping recoverable by construction rather than by convention. D2 and D9 are structurally excluded: the decoder cannot fail to know the ordering, because the encoder wrote it down.

5.9 The Execution Bundle

The artifact that is actually run. Immutable, content-addressed, and the single input to both submission and decoding.

```

interface ExecutionBundle {
  schema_version: "bundle/1";
  id: string;                                // JCS-SHA-256
  envelope_id: string;
  backend_id: string;

  payload: BlobRef;                          // the EXACT bytes submitted
  media_type: string;                        // "application/qpy", "application/json", "text/qasm3", ...

  decode_map: {
    bit_order: "q0_left" | "q0_right";
    registers: Array<{ name: string; width: number; offset: number }>;
    measurement_map: Array<{ qubit: number; clbit: number }>;
    layout: { logical_to_physical: Record<number, number>;
              routing_permutation: number[] } | null;
    result_types: ResultTypeDescriptor[];
  };

  provenance: {
    adapter: { name: string; version: string };
    compiler: { name: string; version: string; optimization_level: number; seed: number };
    capability_fingerprint: string;
    calibration_fingerprint: string | null;
    lowering_proofs: ProofRef[];
  };

  verification: VerificationReport; // §5.11
  metrics: { qubits: number; depth: number; ops: Record<string, number>;
            two_qubit_ops: number };
}

```

`decode_map` is the direct answer to the four worst result defects. `bit_order` is recorded per bundle, so IonQ's `q0_left` and Aer's `q0_right` are both correct and both known (D2). `registers` preserves boundaries (D3). `measurement_map` survives to decode (D5). `layout` is applied in inverse at decode (D9).

For IBM, `payload` is the QPY the worker already produces — the artifact currently computed and thrown away (Fact 2), submitted through the `/v1/providers/ibm/jobs` endpoint that already exists and is already unused. This is less new code than the current path requires.

5.10 Typed results

```
interface ResultSet {
  schema_version: "res/1";
  bundle_id: string;
  backend_id: string;
  data: ResultData[];
  provenance: { decoder_version: string; bit_order: string;
                layout_applied: boolean; synthetic: SyntheticFlag[] };
  raw: BlobRef; // untouched provider response, always retained
}

type ResultData =
| { type: "samples";      register: string; shots: number; bitstrings: string[] }
| { type: "counts";      register: string; shots: number; counts: Record<string, number> }
| { type: "probabilities"; register: string; probabilities: Record<string, number> }
| { type: "quasi";        register: string; quasi: Record<string, number>;
  mitigation: string } // negatives preserved
| { type: "expectation"; observable: string; value: number; variance?: number;
  stderr?: number }
| { type: "gradient";    observable: string; values: number[]; parameters: string[] }
| { type: "statevector"; amplitudes: BlobRef }
| { type: "density_matrix"; matrix: BlobRef }
| { type: "annealing";    samples: Array<{ sample: Record<string, number>; energy: number;
  num_occurrences: number; chain_break_fraction?: number }> }
| { type: "analog";       shots: number; site_measurements: BlobRef }
| { type: "photon_pattern"; patterns: Array<{ modes: number[]; count: number }> };
```

Three rules govern the decoder:

1. **raw is always kept.** Whatever the typed decode produces, the untouched provider response is retained and addressable. No information is destroyed by normalisation.
2. **Synthesis is labelled.** Counts derived from probabilities carry `synthetic: [{ field: "counts", reason: "derived_from_probabilities", method: "largest_remainder" }]`. This is the missing annotation from D4.
3. **Quasi-probabilities are never clipped.** `type: "quasi"` keeps negative mass. Converting a quasi-distribution to counts is *not* a decode operation; if a client wants it, it asks for it explicitly and gets the synthesis flag.

`annealing` carries `energy` and `num_occurrences` because a D-Wave `SampleSet` is not a histogram over bitstrings and cannot be projected into one without losing the objective value.²¹

5.11 The verification portfolio

```

type VerificationStatus =
  | "proved"           // exact equivalence established
  | "checked"         // independent method agreed (DD / ZX / differential simulation)
  | "provider_validated" // provider accepted and validated the artifact
  | "partial"         // some gates proved, others not
  | "unsupported"      // no applicable method for this workload
  | "failed";         // a check actively disagreed → hard stop

interface VerificationReport {
  status: VerificationStatus;
  gates_run: Array<{ gate: string; status: VerificationStatus; detail: string;
                    evidence?: BlobRef }>;
  bound_to: { compiler_version: string; capability_fingerprint: string;
             calibration_fingerprint: string | null };
}

```

Replacing `boolean | null` (D15) with a status makes “we did not check” and “we checked and it is fine” different words, which they are.

Gates, cheapest first:

Gate	Applies to	Method
G1 Parse & typecheck	all	frontend AST + type rules
G2 Resource bounds	all	qubits, depth, ops, batch, expression depth
G3 Target legality	all	<code>satisfies()</code> re-run against the compiled artifact
G4 Structural preservation	gate, dynamic	measurement map, register widths, clbit count
G5 Decomposition unitaries	lowering rules	exact operator comparison, up to global phase
G6 Exact circuit equivalence	≤ ~10–12 qubits	dense operator comparison
G7 Scalable equivalence	larger circuits	MQT QCEC: DD / ZX / alternating ²²
G8 Dynamic-path equivalence	dynamic	per-branch channel comparison
G9 Differential execution	all simulable	independent simulators, statistical agreement
G10 Metamorphic / property	adapters	round-trip, permutation, identity-insertion invariants
G11 Provider schema validation	all	validate before submission, not on rejection

G5 is the one that had to exist and did not. It runs in CI over the whole lowering rule set, not per job. The audit script written for this document (Appendix B) is a working prototype: it recovers each rule’s full unitary by running every computational basis state through the real code path and compares against a reference matrix up to global phase. It found D1 on first run and cleared the other 21 rules. Promoting it to a build gate is roughly a day’s work and closes RC5 permanently.

5.12 The application encoding recipe library

Category 2 from §3.6 — explicit, versioned, opt-in.

```
interface EncodingRecipe {
  id: string; // "amplitude/v1", "angle/v1", "reupload/v2"
  maturity: "stable" | "experimental" | "research";
  inputs: DataSpec;
  estimate(input: DataSpec): { qubits: number; depth: number; two_qubit_ops: number;
                              ancillas: number; caveats: string[] };
  build(input: DataSpec): GateProgram;
}
```

The contract is that `estimate()` runs **before** quoting and its output is shown to the user. `amplitude/v1` returns a depth estimate that is exponential in the qubit count for generic states, with the caveat stated plainly; a user who accepts that has made an informed choice, and one who does not is not silently charged for it. `gram/v1` is `maturity: "research"` and is not selectable in production. Block encoding and QSVT are `estimation`-kind workloads (§5.3) that produce fault-tolerant resource estimates rather than executable jobs, because that is what they honestly are on current hardware.

No recipe is ever selected by the router. Selection is always an explicit field on the request.

5.13 Security model

Every submitted program is hostile input, and the frontends are parsers — historically the highest-risk component in any such system.

- **Quotas at ingest**, enforced before parsing completes: source bytes, token count, AST depth, operation count, expression nesting depth, loop-unroll bound, register count, batch size, and serialised result size. The current 256 KB byte cap ([analyze.ts:5](#)) is the only one that exists today.
- **No filesystem or network includes**. An explicit allow-list of standard includes, resolved from an in-memory table. This makes D17's protection deliberate instead of accidental — a prerequisite for replacing the analysis parser, not an optional hardening.
- **Never accept executable serialisation from users**. QPY, pickle, and LLVM bitcode may be *produced* by the platform and *submitted* to providers; they must never be accepted from a client. QPY deserialisation is code-adjacent and its trust boundary runs one way only.
- **Isolated, deadlined compilation**. Parsing, lowering, and compilation run in workers with CPU and wall-clock limits and no ambient credentials.
- **Signed capability profiles and bundles**. A bundle records the capability fingerprint it was built against; a mismatch at submit time is a hard failure, not a warning.
- **Redaction**. Provider payloads and credentials never reach logs or artifacts. Circuits are user IP and inherit the existing retention and purge path (`purgeCircuitData`).
- **Fix D10 first**. The v2 test-key gap is an authorization defect that predates this work and is not fixed by it. It ships in Phase 0.

5.14 Versioning and provider drift

Every schema in this document carries `schema_version`. Beyond that:

- Capability profiles have a TTL and an `is_stale` flag; a stale profile can serve a quote but cannot authorise a submission.
 - Bundles pin adapter, compiler, capability fingerprint, and calibration fingerprint. A recalibration between compile and submit invalidates the bundle and triggers recompilation rather than a silently degraded run.
 - `provider_schema_version` is recorded per profile, so a provider API change surfaces as a version mismatch on one adapter instead of a decode failure everywhere.
 - Adapters are independently versioned and independently rolled out. This is what makes the strangler migration in Part V possible without a flag day.
-

Part IV — Adversarial review

The architecture in Part III was subjected to three review rounds, each with a different adversary’s brief. Criticisms that survived scrutiny changed the design; the changes are marked ► **Design change**. Criticisms that did not survive are recorded with the reason, so the question is not reopened later.

6. Round 1 — semantic correctness and cross-paradigm completeness

C1.1 — “Up to global phase” is an unsound proof standard for this IR

The criticism. §5.4 admits `ctrl @`, `inv @`, `pow @` modifiers, and §5.3 has a `dynamic` workload whose gates sit inside measurement-conditioned branches. Verification gate G5 proves decomposition rules *up to global phase*. A global phase is only unobservable when the operator is applied unconditionally. Under `ctrl @`, the phase of the controlled block becomes a **relative** phase between the control’s $|0\rangle$ and $|1\rangle$ branches — observable, and wrong.

This is not hypothetical. The current `cu1` rule uses `rz( $\theta/2$ )` where `u1( $\theta/2$ )` is required; the product differs from true `cu1` by $e^{-i\theta/4}$. Standalone that is invisible. Written as `ctrl @ cu1( $\theta$ )`, it is a wrong answer. The same applies to `cu3`, `crz`, and `csx`. Adopting up-to-phase proofs would let the design ship a whole family of D1s.

Verdict: sustained. The proof standard is unsound for the IR being proposed.

► **Design change.** Two-tier proof standard on every lowering rule:

```
interface LoweringRule {
  id: string;
  phase_exact: boolean;           // proved as an exact operator, not up to phase
  proof: ProofRef;
}
```

G5 requires `phase_exact: true` for any rule reachable under a modifier or inside a conditional branch; up-to-phase proofs are permitted only for rules proved unreachable in those positions. The lowering pass refuses to place a non-exact rule under a modifier. Concretely, `cu1`, `cu3`, `crz`, and `csx` must be rewritten with `u1 / p` in place of `rz` on the phase-bearing wires before they can be used under control.

C1.2 — Recording bit order is not the same as fixing it

The criticism. §5.9 stores `bit_order` per bundle and §5.8 claims D2 is “structurally excluded”. It is not. If IonQ’s result is decoded as `q0_left` and Aer’s as `q0_right`, both decodes are *faithful* and the user still receives reversed bitstrings for the same circuit depending on routing. The design records the problem rather than solving it.

Verdict: sustained. This was a real gap and the original wording concealed it.

► **Design change.** The decoder does two things, not one: it reads the source convention from `decode_map.bit_order` and then **normalises to a single declared platform convention** before returning. `ResultSet.provenance` retains the source convention for audit. The platform convention is `q0_right`

(rightmost character is qubit 0), matching Qiskit, Aer, and the overwhelming majority of user expectation, and it is documented in the API contract rather than left implicit. Preservation and normalisation are both required; neither substitutes for the other.

C1.3 — Matching capabilities by instruction *name* recreates D6 one layer up

The criticism. §5.7 checks that a required instruction name appears in the capability’s instruction list. But the whole point of D6 is that the same operation is called `sdg` by IBM, `si` by IonQ, and `ti / ccnot / cnot` elsewhere. String comparison over provider vocabularies is exactly the failure being fixed, relocated into `satisfies()`.

Verdict: sustained.

► **Design change.** Requirements and capabilities are expressed over a canonical **operation identity**, not a name:

```
interface OpId {
  key: string;           // canonical id, e.g. "op:sdg" – derived from the definition
  arity: { qubits: number; params: number };
  definition: UnitaryRef | "opaque"; // reference matrix or parameterised family
}
```

Provider names become an adapter-local rendering (`OpId` → `provider token`) applied in `encode`, and provider capability discovery maps the other way (`provider token` → `OpId`) in `profile`. The `OpId` registry is keyed by definition, so `sdg` and `si` resolve to one identity and the `sx / v` gap of D7 cannot occur — it becomes a missing entry in one rendering table, caught by the adapter’s own conformance test rather than at submission time.

C1.4 — Eight workload kinds is eight times the surface, mostly unbuilt

The criticism. Each variant needs a frontend, requirements extraction, adapter methods, verification gates, and result types. Most will be stubs for a long time, and a stub that appears in a public union advertises capability that does not exist — which is D11 rebuilt deliberately.

Verdict: partially sustained. The union is right; advertising it eagerly is wrong.

► **Design change.** The union is closed in the type system but **gated at the API boundary by `CapabilityProfile.workload_kinds`**. A workload kind is visible to clients only when at least one adapter implements it end to end, including decode and at least one verification gate. Build order is demand-driven: `gate` → `dynamic` → `primitive` → `annealing` → `analog` → `photonic` → `timed` → `estimation`. Kinds beyond `primitive` are explicitly not committed (see C2.5).

C1.5 — “No universal core gate set” removes the basis for pre-compilation metrics

The criticism. §5.4 abolishes the platform-wide core set, so depth and gate counts are only defined *per target after lowering*. But pricing, complexity classification, and the local simulator all consume metrics today, and some of them run before a target is chosen.

Verdict: not sustained — but it exposes a consequence worth stating. Metrics *should* be target-relative; a circuit’s depth on a heavy-hex lattice is genuinely not its depth on an all-to-all machine, and the current code already acknowledges this by repricing after compilation (`pipeline.ts:25`). The IR carries **source metrics**

(structural, target-independent: qubit count, statement count, two-qubit statement count) for admission control and coarse classification, and **compiled metrics** live on the bundle for pricing. What is removed is the pretence that one normalised gate count is meaningful across targets.

The real consequence is that a *binding* quote requires a compile, which is principle 2 and is addressed on cost grounds in C2.1.

C1.6 — Writing an OpenQASM 3 frontend is a months-long project

The criticism. §5.4 requires a parser and typechecker for a language with classical types, control flow, timing, subroutines, and pulse-level blocks. That is not a phase; that is a product.

Verdict: sustained as a risk, resolved by an existing asset. The repository already depends on `qasm-ts@2.1.4`, which ships both `qasm2` and `qasm3` parsers — and `grep` confirms it is **not referenced anywhere** in `src/`, `cli/`, `sdk/`, or `tests/`. The OpenQASM 3 frontend is already in `node_modules`, already in `package.json`, and unused, while `analyze.ts` performs regex downgrades beside it.

► **Design change.** The frontend is an *adaptation* of `qasm-ts` into the semantic IR, not a parser written from scratch. It carries a mandatory conformance suite, because a vendored parser is exactly the dependency class that produced this document's other findings — `quantum-circuit` disagrees with Qiskit on what it accepts, and `quantum-computer-js` *silently deletes* operations outside its known set, a hazard the current code works around with the `LOCAL_OPTIMIZER_GATES` guard ([transpiler.ts:32](#)). The parser is adopted with a differential test against Qiskit's parser, not on trust.

7. Round 2 — cost, complexity, performance, and migration feasibility

C2.1 — Compiling N candidates multiplies compiler load past the breaking point

The criticism. Principle 2 says compile before quoting. §5.1 shows a compile fan-out over candidates. But `/api/v2` already accepts up to 25 executions per group, each of which is a full remote transpile, throttled to `EXECUTION_FANOUT_LIMIT = 4` ([v2-service.ts:24](#)). Multiplying by a candidate set of 6–9 backends yields up to 225 Qiskit transpiles for one request against a single worker. Latency and cost become unacceptable and the worker becomes a hard availability dependency for quoting.

Verdict: sustained. This is the strongest objection in the review.

► **Design change.** Three mechanisms, in order of effect:

1. **Prune before compiling.** `satisfies()` is pure, local, and runs against cached capability profiles. It eliminates incompatible candidates for free — something the current architecture cannot do at all, since its only filter is qubit count. Typical candidate sets shrink from 6–9 to 2–3 before any compile.
2. **Tiered compilation.** Compile the primary plus `K` failover candidates eagerly (`K = 2` by default, configurable to 0). Remaining candidates compile **lazily, on failover**. A quote is binding for the primary and indicative for the rest, and the API says which is which.
3. **Exact bundle caching.** Cache on `(envelope_id, backend_id, capability_fingerprint, compiler_version, optimization_level, seed)`. Because `envelope_id` is a canonical hash (§3.4), this is an exact cache, not a heuristic one. The dominant v2 workload — one circuit, many executions differing only in shots or routing mode — hits this cache on every execution after the first. Shots are not a compile input, so a 25-execution group over one circuit performs **one** compile per candidate backend, not 25.

Net effect: worst case falls from ~ 225 compiles to ~ 3 , and the common case to $\sim 1-3$.

C2.2 — This is a core rewrite with no user-visible feature

The criticism. A large refactor of the product's most critical path, delivering nothing a customer can see, is the classic profile of a branch that never lands.

Verdict: sustained as a process risk.

► **Design change.** No phase is allowed to be a pure refactor. Each phase in Part V ships either a fixed defect or a new capability, and each runs in **shadow mode** before switching: the new path executes alongside the old, outputs are compared, divergences are logged, and behaviour does not change until divergence is zero on real traffic. Every phase has an explicit exit criterion and an independent rollback. Phase 0 alone fixes two critical defects in days.

C2.3 — Canonical hashing over floating-point parameters is fragile

The criticism. `envelope_id` hashes user-supplied angles. JCS constrains numbers to IEEE-754 doubles.²³ A client sending `0.1+0.2` and one sending `0.30000000000000004` produce different ids for the same intended circuit, and a client using a higher-precision type loses precision at the boundary.

Verdict: partially sustained — the behaviour is correct but was not stated.

► **Design change.** Documented explicitly: `envelope_id` is the identity of the **submitted program as written**, not of a semantic equivalence class. Parameters normalise to IEEE-754 doubles at ingest with JCS/ECMAScript decimal rendering; symbolic parameters are carried as string expressions and are not folded. Semantic deduplication, if ever wanted, is a separate index built on top and is explicitly out of scope. The current `evaluateParam` already rejects non-finite values ([dialects.ts:90](#)), which satisfies the I-JSON constraint.

C2.4 — The test matrix is combinatorial

The criticism. $8 \text{ workload kinds} \times 6+ \text{ backends} \times 11 \text{ verification gates}$ is a matrix nobody maintains.

Verdict: not sustained. The matrix is sparse and *declared*: `CapabilityProfile.workload_kinds` enumerates the (kind, backend) pairs that exist, and each verification gate carries an applicability predicate over workload kind. Tests are generated from the declarations, so an untested combination is one that no profile claims. The matrix is bounded by what is advertised, which C1.4 already gates.

C2.5 — The plan is too large for the team that has to build it

The criticism. Eight phases, eight workload kinds, eleven verification gates, a blob store, a capability system, and an adapter framework is not a quarter of work.

Verdict: sustained, and it deserves a plain answer rather than a schedule.

► **Design change.** The phases are ordered by value density and are independently shippable, and the document commits to that ordering rather than to the whole:

- **Phase 0** (days): fixes a wrong gate and an authorization hole. Standalone value.
- **Phases 1–4** (the real investment): envelope, semantic frontend, adapters, exact artifacts. This is where every remaining Critical and High defect closes.

- **Phases 5–7** (demand-gated): dynamic/timed, analog/annealing/photonic, recipes and estimation. **These may never be built.** The architecture is correct without them; the workload union is designed so they can be added without redesign, which is the entire benefit of choosing a union now. Building the union does not commit to filling it.

Stating this openly is better than an implied promise of all eight phases.

C2.6 — Vended parsers cannot be trusted during a shadow migration

The criticism. Shadow mode compares old against new. But the *old* path’s own components are known to corrupt: `quantum-computer-js` “silently DELETES any other operation from the circuit”, per the comment guarding it at [transpiler.ts:29](#). Comparing against a known-faulty oracle produces divergences that are the *new* path being right.

Verdict: sustained.

► **Design change.** Shadow divergences are **triaged, not counted**. Each divergence is classified as `new_correct`, `old_correct`, or `both_wrong`, adjudicated against an independent reference (Qiskit for parsing and compilation; exact simulation for semantics) rather than against the old path. The exit criterion is “no `old_correct` divergences”, not “no divergences”. The local optimizer is retired in Phase 2 rather than carried, precisely because its deletion behaviour makes it useless as an oracle.

8. Round 3 — security, versioning, provider drift, and failure recovery

C3.1 — Cross-tenant content addressing is an information-disclosure oracle

The criticism. §5.9 makes blobs content-addressed by hash. If the store is global, deduplication is observable: an attacker who guesses a competitor’s circuit can submit it and detect, through storage accounting or latency, that the blob already existed — confirming another tenant ran that exact circuit. Circuits are user IP. Deduplication across a trust boundary is a confirmation oracle.

Immutability also complicates the existing deletion path: `purgeCircuitData` scrubs a circuit on deletion ([v2-service.ts](#)), and a shared immutable blob cannot simply be deleted on one tenant’s request.

Verdict: sustained. This is the most serious finding of the review.

► **Design change.** Content addressing is **scoped per organization**. The storage key is `H(organization_id || content)`, so identical content in two organizations produces two independent blobs and no cross-tenant inference is possible. Within an organization, blobs are reference-counted; purge decrements and deletes at zero, preserving the existing retention semantics. Cross-tenant deduplication is **rejected outright**, at a storage cost that is negligible next to the disclosure risk.

C3.2 — Signing platform-generated profiles with a platform key is theatre

The criticism. §5.13 says capability profiles and bundles are “signed or hashed”. If the platform both produces and verifies the signature, it defends against nothing that a hash does not — certainly not against a platform-side bug, which is the actual threat model established by Part I.

Verdict: sustained.

► **Design change.** Downgraded to **fingerprint and pin**. Profiles and bundles carry JCS hashes; bundles pin the fingerprints they were built against; a mismatch is a hard failure. Signing is reserved for the one case where it means something — profiles crossing a trust boundary, such as a customer-operated worker — and is deferred until that exists. No cryptography is deployed whose threat model cannot be stated.

C3.3 — Failing closed on drift will cause mass job failure

The criticism. §5.14 invalidates a bundle when its calibration fingerprint changes. IBM recalibrates its machines frequently. Under this rule a routine recalibration invalidates every queued bundle, forcing mass recompilation and potentially cascading failures at exactly the moment the fleet is busiest.

Verdict: sustained. The original design conflated two different kinds of drift.

► **Design change.** They are separated, and only one is load-bearing for legality:

Drift	Examples	Effect
Capability drift	gate set, coupling map, control-flow support, limits	Hard invalidate. The artifact may be illegal. Recompile before submit.
Calibration drift	error rates, gate durations, readout fidelity	Soft. The artifact remains legal. Affects scoring and requoting only; recorded in provenance.

Only capability drift blocks submission. Calibration drift updates routing scores and, if the change moves price beyond the quote’s tolerance, triggers a requote — never an invalidation.

C3.4 — A buggy verifier becomes a global outage

The criticism. §5.11 makes `failed` a hard stop. An equivalence checker with a false positive — or a checker bug after an upgrade — would block every job it touches.

Verdict: sustained.

► **Design change.** `failed` blocks **that bundle, not that job**. The router falls through to the next compiled candidate and records the failure with its evidence. Verification methods are individually versioned and individually disableable by configuration with an audit record. A fleet-wide spike in `failed` is monitored as an anomaly in the *verifier*, since a checker that rejects everything is far more likely to be broken than a fleet that became wrong simultaneously.

C3.5 — Idempotency keyed on bundle hashes breaks retries

The criticism. If a provider idempotency token derives from `bundle_id`, then any retry that recompiles — after a capability drift, a compiler upgrade, or a verification fallback — produces a new bundle, a new id, a new token, and therefore a **duplicate provider submission**, billed twice. The current code already derives Braket’s `clientToken` and IonQ’s job name from the job id ([execution.ts:61, 182](#)), which is the correct shape and must not be regressed.

Verdict: sustained.

► **Design change.** Provider idempotency keys derive from `(envelope_id, backend_id, execution_position, attempt_number)` and explicitly **not** from `bundle_id`. The bundle may change across a retry; the idempotency identity must not. `bundle_id` is recorded in provenance so the operator can see *which* artifact a given attempt actually ran.

C3.6 — The recipe library is a scope-creep vector

The criticism. Once the platform ships amplitude encoding and fault-tolerant estimation, it is in the algorithms business, and the encoding layer's proof obligations get diluted by a much larger and softer surface.

Verdict: sustained as a governance risk.

► **Design change.** A hard architectural boundary: recipes are **pure functions** `DataSpec → GateProgram` with no access to routing, capability profiles, execution, or billing. They sit *above* the envelope and produce input to it. The router never selects one. The entire library can be deleted without touching a line of the encoding layer, and that property is enforced by module boundaries and a dependency lint, not by convention.

9. Rejected alternatives

Recorded so they are not relitigated.

Alternative	Why rejected
One universal gate IR that every provider lowers from	This is the current design’s premise, and D6/D7 are its consequences. No universal core set exists; <code>CORE_G</code>
QIR as the canonical IR	Base Profile forbids mid-circuit measurement and dynamic control flow and requires static qubit counts; ²⁴ no
Protobuf for the envelope	Explicitly non-canonical serialisation; unusable as a hashing substrate. ²⁵
Normalise all results to <code>{counts, probabilities, shots}</code>	Cannot represent energies, expectations, gradients, quasi-distributions, per-register bit arrays, or photon pa
A hand-written OpenQASM 3 parser	<code>qasm-ts</code> is already a dependency and already unused; adapt and conformance-test it instead.
Compile every candidate eagerly	~225 transpiles per worst-case v2 request (C2.1). Replaced by prune → tiered → cached.
Global cross-tenant content-addressed store	Deduplication oracle across a trust boundary (C3.1). Scoped per organization instead.
Platform-signed capability profiles	No threat model the hash does not already cover (C3.2). Deferred to cross-trust-boundary distribution.
Invalidate bundles on calibration change	Mass failure on routine recalibration (C3.3). Only capability drift invalidates.
Keep equivalent: <code>boolean \</code> <code>null</code>	Conflates “not attempted”, “not provable”, and “crashed” (D15).

Part V — Migration

10. Strangler rollout

Every phase runs in shadow before it switches, has an explicit exit criterion, and rolls back independently. Divergences are triaged against an independent reference, never against the old path (C2.6). Sizes are relative effort, not calendar commitments.

Phase 0 — Stop the bleeding (*days; no architecture*)

Nothing here depends on the rest of the plan.

1. **Fix `ch`** — swap the `ry` signs at [dialects.ts:156](#).
2. **Add the G5 unitary gate to CI** — promote the audit harness (Appendix B) to a test that proves all 22 lowering rules against reference operators. This is what makes fix 1 permanent.
3. **Close the v2 test-key hole (D10)** — apply `backendsForPrincipal` and `assertTargetAllowed` in `createExecutionGroup`, matching v1.
4. **Correct the misleading comment** in `dialects.ts` about what is verified and where.
5. **Label synthetic counts (D4)** and stop clipping quasi-probabilities.

Exit: all 22 rules proved in CI; a test key provably cannot reach a QPU on v2; no unlabelled synthetic counts.

Phase 1 — Envelope and artifact model (*no behaviour change*)

Introduce `ExecutionEnvelope`, JCS canonical hashing, org-scoped content-addressed blobs (C3.1), `ExecutionBundle`, `VerificationReport`, and the typed `ResultSet` — written alongside the existing path and populated in shadow. Existing responses continue to be produced by the old code.

Exit: every job produces a valid envelope and bundle; hashes are stable across restarts and re-serialisation; purge semantics verified against the existing retention tests.

Phase 2 — Semantic frontend (*retires the regex layer*)

Adapt `qasm-ts` into `GateProgram` for OpenQASM 2 and 3. Retire `toOpenQasm2`'s regex downgrade and keyword blocklist (D12), the text-splice `expandDialects` (D1's habitat), and the silently-deleting local optimizer (D14, C2.6). Make the include policy explicit before the parser changes (D17). Ship differential tests against Qiskit's parser.

Exit: no `old_correct` divergences on replayed production traffic; OpenQASM 3 programs using `for / if / input` parse successfully (execution still gated by capability); include policy is enforced by an allow-list rather than by parser accident.

Phase 3 — Capability profiles and satisfaction routing

Adapters produce `CapabilityProfile`s from live provider data with static fallbacks explicitly labelled. Replace `compatibility()` with `satisfies()` over `OpId` (C1.3). Fix `/api/v2/backends` to report real per-backend capability (D11).

Exit: every rejection names the unmet requirement; no backend advertises a capability its adapter cannot encode; the `sx / v` class of mismatch (D7) is caught by adapter conformance tests rather than at submission.

Phase 4 — Real adapters with exact artifacts

Implement `discover → profile → validate → lower → compile → encode → submit → poll → cancel → decode` for QCI Aer, IBM, IonQ, and Braket.

- **IBM** submits the **QPY** already produced by the worker, via the `/v1/providers/ibm/jobs` endpoint that already exists and is currently unreachable — closing Fact 2 and D8, and honouring the routed backend rather than an environment variable.
- **IonQ** carries the measurement map (D5) and maps `OpId`s to QIS names including `v / vi` (D7).
- **Braket** emits device-declared gate names (D6) and gains result-type pragmas.
- **Decoders** apply the layout inverse (D9), preserve registers (D3), and normalise bit order to the platform convention while recording the source (C1.2, D2).

Exit: submitted bytes are byte-identical to the hashed bundle payload for every adapter; decode round-trips register structure on multi-register circuits; a differential test shows the same circuit returns the same bitstrings on Aer, IonQ, and Braket.

Phase 5 — Compile-before-quote

Prune with `satisfies()`, compile primary + `K` failover candidates, cache on the envelope hash (C2.1). Failover targets become pre-proven rather than hopeful.

Exit: p95 quote latency within budget; measured compile count per v2 group at or below the candidate count, not the execution count.

Phase 6 — Dynamic and timed workloads (*demand-gated*)

`dynamic` and `timed` variants, mid-circuit measurement and feedback, G8 path equivalence.

Phase 7 — Analog, annealing, photonic, recipes (*demand-gated*)

One workload family at a time, each with its own adapter, decoder, and result type. Photonic backends stop being permanently unavailable and become the correct target for `photonic` workloads. The recipe library (§5.12) and `estimation` land here, behind the module boundary of C3.6.

Phases 6 and 7 are explicitly not committed (C2.5). The architecture is complete and correct without them.

11. What success looks like

Measure	Today	Target
Lowering rules with a machine-checked operator proof	0 of 22	22 of 22, enforced in CI
Same circuit, same bitstring order across backends	no	yes, normalised and recorded
Bytes submitted == bytes verified	no (IBM reconstructs)	yes, hash-checked
Backends whose advertised capability matches their encoder	0	all
Result kinds representable	3	11
Verification outcomes distinguishable	2 (true , null)	6
Test key can reach a QPU on v2	yes	no

Part VI — Appendices

Appendix A — Full unitary audit of the lowering rules

Every decomposition in `src/lib/qrouter/dialects.ts` was reconstructed as a full operator by running each computational basis state through `expandDialects` and the vendored simulator, then compared against an independently constructed reference matrix, up to global phase. Reported value is $\max |U - e^{i\phi} \cdot U_{\text{ref}}|$ over all entries.

Rule	Reference	Max error	Verdict
<code>sx</code>	$\frac{1}{2}[[1+i, 1-i], [1-i, 1+i]]$	0.00	pass
<code>sxdg</code>	<code>sx†</code>	0.00	pass
<code>prx(θ, ϕ)</code>	IQM: $\exp(-i\theta/2 \cdot (\cos \phi \cdot X + \sin \phi \cdot Y))$	$1.1e-16$	pass
<code>gpi(t)</code>	IonQ, turns: $[[0, e^{-i\phi}], [e^{i\phi}, 0]]$	$1.2e-13$	pass
<code>gpi2(t)</code>	IonQ, turns	$9.4e-14$	pass
<code>rxz(θ)</code>	$\exp(-i\theta/2 \cdot Z \otimes X)$	$2.2e-16$	pass
<code>rxx(θ)</code>	$\exp(-i\theta/2 \cdot X \otimes X)$	$4.4e-16$	pass
<code>ryy(θ)</code>	$\exp(-i\theta/2 \cdot Y \otimes Y)$	$4.0e-14$	pass
<code>rzz(θ)</code>	$\exp(-i\theta/2 \cdot Z \otimes Z)$	0.00	pass
<code>zz(t)</code>	IonQ, turns	$6.2e-14$	pass
<code>ecr</code>	Qiskit ECR, control = $q[0]$	$6.3e-13$	pass
<code>ms(ϕ_0, ϕ_1)</code>	IonQ MS, default $\theta = 0.25$ turns	$1.1e-13$	pass
<code>ms(ϕ_0, ϕ_1, θ)</code>	IonQ MS	$1.3e-13$	pass
<code>cu1(θ)</code>	$\text{diag}(1, 1, 1, e^{i\theta})$	$2.2e-16$	pass
<code>iswap</code>	standard iSWAP	$2.2e-16$	pass
<code>xy(θ)</code>	pyQuil XY	$2.1e-14$	pass
<code>crz(θ)</code>	controlled-RZ	$5.6e-17$	pass
<code>cy</code>	controlled-Y	0.00	pass
<code>ch</code>	controlled-H	1.41	FAIL
<code>ch</code>	controlled- $(X-Z)/\sqrt{2}$ (<i>what it actually is</i>)	$3.2e-13$	— confirms the defect
<code>cu3(θ, ϕ, λ)</code>	controlled-U3	$1.2e-16$	pass
<code>csx</code>	controlled- $\sqrt{X}$	$4.5e-13$	pass

21 of 22 rules are correct up to global phase. `ch` is wrong. Note that “up to global phase” is the standard this audit used and, per C1.1, is **not** sufficient for rules used under `ctrl @` — `cu1`, `cu3`, `crz`, and `csx` pass here but require exact re-proof before modifier support.

Appendix B — Reproducing the findings

Everything in Part I was produced against commit `1503057` with a clean worktree, `npm test` green at 232 passing.

The `ch` defect, end to end. Analyse and simulate through the platform's own entry points:

```
import { analyzeCircuit } from "@lib/qrouter/analyze";
import { simulateCircuit } from "@lib/qrouter/simulator";

const src = `OPENQASM 2.0;
include "qelib1.inc";
qreg q[2];
creg c[2];
x q[0];
h q[1];
ch q[0],q[1];
measure q -> c;`;

simulateCircuit(analyzeCircuit(src, "openqasm2"), 20000).probabilities;
// shipped: { "11": 1 }
// correct: { "01": 1 }
```

The full unitary audit (G5 prototype). For each rule, recover the operator column by column by preparing every computational basis state with `x` gates, running the *expanded* program through the vendored simulator, and reading the resulting state vector; then compare to a reference matrix after dividing out a single global phase taken from the largest-magnitude reference entry. The bit convention must be probed empirically first — `x q[0]` on `qreg q[2]` lands on state index 1, so `q[0]` is the **least** significant bit of the state index, and two-qubit reference matrices written with `a` as the high tensor factor must be re-embedded accordingly. Getting this wrong produces false positives on every asymmetric gate, which is how the first run of this audit behaved before the convention was pinned down.

The result-decoding defects. Call `normalizeProviderResult` directly:

```
normalizeProviderResult("b", { counts: { "0 11": 50, "01 1": 50 } }, 100);
// → { counts: { "011": 50 }, shots: 50 }           // D3: merged, half the shots lost

normalizeProviderResult("b", { quasiDistribution: { "00": 1.2, "11": -0.2 }, shots: 1000 }, 1000);
// → { counts: { "00": 1200, "11": 0 } }           // D4: 1200 counts from 1000 shots

normalizeProviderResult("b", { probabilities: { "10": 0.5, "3": 0.5 }, shots: 100 }, 100);
// → keys "10" and "11"                           // decimal 10 read as the bitstring "10"
```

The IonQ measurement drop. `qasm2ToIonqCircuit` on a 3-qubit circuit measuring `q[0]→c[0]` and `q[2]→c[1]` emits two gate objects and no measurement information at all.

The v2 authorization gap. A call-site census — `grep -rn "backendsForPrincipal\|assertTargetAllowed" src/` — returns hits under `src/app/api/v1/` only.

Appendix C — Defect index

ID	Defect	Severity	Root cause	Phase
D1	ch decomposes to the wrong unitary	Critical	RC1 + RC5	0
D2	Cross-provider bit-order divergence	Critical	RC4	4
D10	Test keys can reach a QPU on v2	Critical	— (authz)	0
D3	Register boundaries destroyed; counts merged	High	RC4	4
D4	Quasi-probabilities clipped; synthetic counts unlabelled	High	RC4	0
D5	IonQ encoder drops the measurement map	High	RC1	4
D6	Braket receives non-Braket gate names	High	RC2	4
D7	Advertised capability $\neq$ encoder capability	High	RC2	3
D8	IBM ignores the routing decision	High	RC3	4
D9	Layout and routing permutation dropped	High	RC3/RC4	4
D11	Identical capabilities advertised for all backends	Medium	RC2	3
D12	OpenQASM 3 handled by regex + keyword blocklist	Medium	RC1	2
D13	Compiler response spread in unvalidated	Medium	RC3	1
D14	Dialects do not recurse into user gate bodies	Medium	RC1	2
D15	equivalent: boolean \ null is ambiguous	Medium	RC5	1
D16	SQLite connections never closed	Low	— (ops)	1

ID	Defect	Severity	Root cause	Phase
D17	Filesystem <code>include</code> reaches the worker parser	Low*	— (sec)	2

* Currently non-exploitable — blocked incidentally by the vendored parser, not by policy. Must become explicit before Phase 2 replaces that parser.

References

1. IonQ, direct API submission — output keys are big-endian integers with the leftmost bit corresponding to qubit 0 — <https://docs.ionq.com/guides/direct-api-submission>↗
2. IBM Quantum, primitive input/output (`PrimitiveResult` , `DataBin` , per-register `BitArray`) — <https://quantum.cloud.ibm.com/docs/en/guides/primitive-input-output>↗
3. Amazon Braket, supported OpenQASM features, result-type pragmas, verbatim boxes, and LocalSimulator-only features — <https://docs.aws.amazon.com/braket/latest/developerguide/braket-openqasm-supported-features.html>↗
4. IonQ API v0.4, create job — QIS and native gate sets, multi-circuit limits — <https://docs.ionq.com/api-reference/v0.4/jobs/create-job>↗
5. IBM Quantum, transpilation and the backend Target/ISA requirement — <https://quantum.cloud.ibm.com/docs/en/guides/transpile>↗
6. OpenQASM 3.1 specification — <https://openqasm.com/versions/3.1/>↗
7. Amazon Braket, supported OpenQASM features, result-type pragmas, verbatim boxes, and LocalSimulator-only features — <https://docs.aws.amazon.com/braket/latest/developerguide/braket-openqasm-supported-features.html>↗
8. QIR Base Profile — https://github.com/qir-alliance/qir-spec/blob/main/specification/profiles/Base_Profile.md↗
9. IBM Quantum, transpilation and the backend Target/ISA requirement — <https://quantum.cloud.ibm.com/docs/en/guides/transpile>↗
10. IonQ API v0.4, create job — QIS and native gate sets, multi-circuit limits — <https://docs.ionq.com/api-reference/v0.4/jobs/create-job>↗
11. Amazon Braket, supported OpenQASM features, result-type pragmas, verbatim boxes, and LocalSimulator-only features — <https://docs.aws.amazon.com/braket/latest/developerguide/braket-openqasm-supported-features.html>↗
12. IBM Quantum, primitive input/output (`PrimitiveResult` , `DataBin` , per-register `BitArray`) — <https://quantum.cloud.ibm.com/docs/en/guides/primitive-input-output>↗
13. IonQ, direct API submission — output keys are big-endian integers with the leftmost bit corresponding to qubit 0 — <https://docs.ionq.com/guides/direct-api-submission>↗
14. Amazon Braket, supported OpenQASM features, result-type pragmas, verbatim boxes, and LocalSimulator-only features — <https://docs.aws.amazon.com/braket/latest/developerguide/braket-openqasm-supported-features.html>↗
15. IonQ API v0.4, create job — QIS and native gate sets, multi-circuit limits — <https://docs.ionq.com/api-reference/v0.4/jobs/create-job>↗

16. D-Wave, SampleSet semantics (`sample` , `energy` , `num_occurrences` , `chain_break_fraction`) — <https://docs.dwavequantum.com/en/latest/concepts/samplesets.html>↗
17. RFC 8785, JSON Canonicalization Scheme — <https://www.rfc-editor.org/rfc/rfc8785.html>↗
18. Protocol Buffers, serialisation is not canonical — <https://protobuf.dev/programming-guides/serialization-not-canonical/>↗
19. MQT QCEC, equivalence checking (decision diagrams, ZX-calculus, alternating, simulation) — <https://mqt.readthedocs.io/projects/qcec/en/latest/>↗
20. QIR Base Profile — https://github.com/qir-alliance/qir-spec/blob/main/specification/profiles/Base_Profile.md↗
21. D-Wave, SampleSet semantics (`sample` , `energy` , `num_occurrences` , `chain_break_fraction`) — <https://docs.dwavequantum.com/en/latest/concepts/samplesets.html>↗
22. MQT QCEC, equivalence checking (decision diagrams, ZX-calculus, alternating, simulation) — <https://mqt.readthedocs.io/projects/qcec/en/latest/>↗
23. RFC 8785, JSON Canonicalization Scheme — <https://www.rfc-editor.org/rfc/rfc8785.html>↗
24. QIR Base Profile — https://github.com/qir-alliance/qir-spec/blob/main/specification/profiles/Base_Profile.md↗
25. Protocol Buffers, serialisation is not canonical — <https://protobuf.dev/programming-guides/serialization-not-canonical/>↗